

 | Emmanuel Rachelson | <https://supaerodatascience.github.io/machine-learning/>

Support Vector Machines,
the Bias-Variance tradeoff
and a short introduction to kernel theory

SVM is a powerfull method, used mainly for **Classification**. As we will see, we use usually a combination of two steps :

- Draw a line that sits as far as possible from the data points (Support Vector Machines)
- Send all data points in a higher dimension space where they are linearly separable (Kernel trick)



1. [A geometrical approach to Machine Learning: margin maximization](#)

1. [The notion of margin](#)
 2. [Optimal separating hyperplanes](#)
 3. [Support vectors and sparsity](#)
 4. [The non-linearly separable case](#)
 5. [Making predictions](#)
2. [Support Vector Machines in scikit-learn](#)
 3. [When using linear separators makes no more sense](#)
 4. [A word on the bias-variance tradeoff](#)
 5. [The kernel trick](#)
 1. [The intuition of mapping to higher dimensions](#)
 2. [The kernel trick](#)
 3. [Positive definite kernels](#)
 6. [SVMs and kernels](#)
 7. [What about other uses?](#)
 8. [Examples](#)
 1. [Spam or ham?](#)
 2. [NIST](#)

In a nutshell:

- Support Vector Machines try to separate data by maximizing a geometrical margin
- They are computed offline
- They offer a sparse, robust to class imbalance, and easy to evaluate predictor
- Kernels are a way of enriching (lifting) the data representation so that it becomes linearly separable
- SVMs + kernels offer a versatile method for classification, regression and density estimation
- [Documentation in scikit-learn](<http://scikit-learn.org/stable/modules/svm.html>)

✓ 1. A geometrical approach to Machine Learning

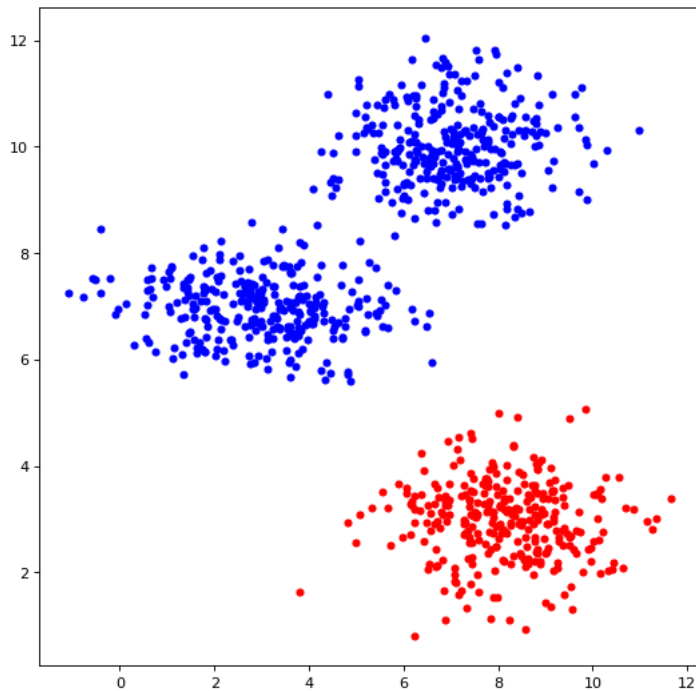
1.1 The notion of margin

Suppose we are given the dataset $\{(x_i, y_i)\}$ below.

```
import numpy as np
%matplotlib inline
#%matplotlib notebook
import matplotlib.pyplot as plt
fig_size=(8, 8)

res = np.loadtxt("sep_lin.csv", delimiter=',')
X = res[:,0:-1]
y = res[:, -1].astype(int)
Xblue = X[y==-1]
Xred = X[y==1]

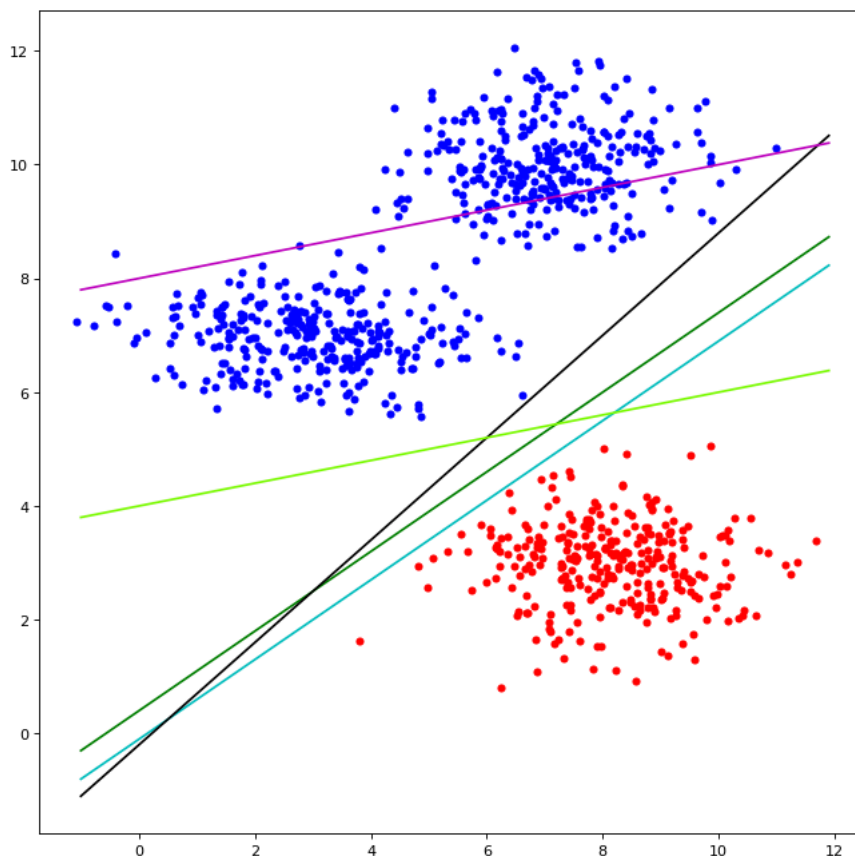
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b',s=20)
plt.scatter(Xred[:,0],Xred[:,1],c='r',s=20);
```



Question (collective answer)

- What constitutes a "good" separating line between red and blue dots? We want to maximise the minimal distance between the points
- What about the lines below?

```
fig_size=(10,10)
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b',s=20)
plt.scatter(Xred[:,0],Xred[:,1],c='r',s=20)
XX = np.arange(-1.,12.,0.1)
YY = 0.7*XX+0.4
plt.plot(XX,YY,c='g')
YY = 0.7*XX-0.1
plt.plot(XX,YY,c='c')
YY = 0.9*XX-0.2
plt.plot(XX,YY,c='k')
YY = 0.2*XX+4
plt.plot(XX,YY,c='lawngreen')
YY = 0.2*XX+8
plt.plot(XX,YY,c='m');
```



The best separating hyperplane is the one that maximizes the distance to the closest points.

Question

Suppose w is the hyperplane's normal vector, w_0 is its intercept, and $\|w\| = 1$. What is the expression of this distance?

▼ Ready to see the answer? (click to expand)

Let's write M this distance and call it **the margin**.

$$M = \min_i y_i (w^T x_i + w_0)$$

Question

What is the margin of the dark green line in the example above?

To answer this question, recall that given a hyperplane of equation $w^T x + w_0 = 0$, with $\|w\| = 1$, the (signed) distance between x_i and the hyperplane is precisely $w^T x_i + w_0$.

Commencez à coder ou à [générer](#) avec l'IA.

```
w_line = np.array([0.9, -1.]) # From the equation Y = 0.9*X - 0.2, which is 0.9*X - Y - 0.2 = 0
w0_line = -0.2

# Normalize w such that ||w|| = 1
norm_w = np.linalg.norm(w_line)
w_normalized = w_line / norm_w
w0_normalized = w0_line / norm_w

# Combine Xblue and Xred back into a single X and y for calculation
X_combined = np.vstack((Xblue, Xred))
y_combined = np.concatenate((-np.ones(Xblue.shape[0]), np.ones(Xred.shape[0])))

# Calculate the signed distance for each point
signed_distances = y_combined * (np.dot(X_combined, w_normalized) + w0_normalized)

# The margin is the minimum of these signed distances
margin_dark_green_line = np.min(signed_distances)

print(f"Le vecteur normal (w) de la ligne verte foncée est : {w_line}")
print(f"L'ordonnée à l'origine (w0) de la ligne verte foncée est : {w0_line}")
print(f"La marge de la ligne verte foncée est : {margin_dark_green_line}")
```

Le vecteur normal (w) de la ligne verte foncée est : [0.9 -1.]
L'ordonnée à l'origine (w0) de la ligne verte foncée est : -0.2

La marge de la ligne verte foncée est : 0.1451723776744595

1.2 Optimal separating hyperplanes

We want to find the hyperplane that maximizes the margin, so:

$$\begin{aligned} \max_{M, w, w_0} \quad & M \\ y_i(w^T x_i + w_0) & \geq M, \forall i \in [1, N] \\ \|w\| & = 1 \end{aligned}$$

Let's use some optimization routines for that.

Exercise

The `fmin_slsqp` function from `scipy.optimize` solves constrained non-linear optimization problems by sequential least squares programming. It takes as inputs:

- a function `func` of variable vector `x`,
- an initial guess `x0` for `x`,
- a function `eq_const` that returns a vector of values corresponding to the equality constraints `eq_const(x)==0`,
- a function `ineq_const` that returns a vector of values corresponding to the inequality constraints `ineq_const(x)>=0`.

```
# %load solutions/code2.py
### WRITE YOUR CODE HERE
# If you get stuck, uncomment the line above to load a correction in this cell (then you can execute this code).

import scipy.optimize as sopt

def func(x):
    """x = (M,w_1,w_2,b)"""
    return -x[0]

def ineq_constr(x):
    """x = (M,w_1,w_2,b)"""
    return y*(x[1]*X[:,0] + x[2]*X[:,1] + x[3]*np.ones(X.shape[0])) - x[0]*np.ones(X.shape[0])

def eq_const(x):
    return x[1]*x[1]+x[2]*x[2]-1

x0 = np.array([0.,0.7,-1.,0.4])

res = sopt.fmin_slsqp(func, x0, f_eqcons=eq_const, f_ineqcons=ineq_constr)

print(res)
print("margin =", res[0])
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b')
plt.scatter(Xred[:,0],Xred[:,1],c='r')
XX = np.arange(-1.,12.,0.1)
YY = -(res[1]*XX+res[3])/res[2]
plt.plot(XX,YY,c='g')
YY = -(res[1]*XX+res[3]+res[0])/res[2]
plt.plot(XX,YY,'g--')
YY = -(res[1]*XX+res[3]-res[0])/res[2]
plt.plot(XX,YY,'g--');
```

We can simplify the optimization problem above.

Exercise:

Use the variable change $w' = \frac{w}{M}$ and $w'_0 = \frac{w_0}{M}$ to get rid of M in the optimization problem above.

► Ready to see the answer? (click to expand)

Let's check we obtain the same result.

```
def func(w):
    """w=(w_0,w_1,w_2)"""
    return .5*(w[1]**2 + w[2]**2)

def ineq_constr(w):
    """w=(w_0,w_1,w_2)"""
```

```

return y*(w[1]*X[:,0] + w[2]*X[:,1] + w[0]*np.ones(X.shape[0])) - np.ones(X.shape[0])

x0 = np.array([0.4,0.7,-1.])
res = sopt.fmin_slsqp(func, x0, f_ieqcons=ineq_constr)
print(res)
margin = 1. / np.linalg.norm(np.array([res[1],res[2]]))
print("margin =", margin)
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b')
plt.scatter(Xred[:,0],Xred[:,1],c='r')
XX = np.arange(-1.,12.,0.1)
YY = -(res[1]*XX+res[0])/res[2]
plt.plot(XX,YY,c='g')
YY = -(res[1]*XX+res[0]+1.)/res[2]
plt.plot(XX,YY,'g--')
YY = -(res[1]*XX+res[0]-1.)/res[2]
plt.plot(XX,YY,'g--');

```

✓ 1.3 Support Vectors and sparsity

This is a quadratic programming problem. Let's try to solve it analytically.

We can introduce the Lagrange multipliers α_i . Then the Lagrangian of the problem is:

$$L(w, w_0, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^N \alpha_i (y_i (w^T x_i + w_0) - 1)$$

The solution must respect the first order Karush-Kuhn-Tucker conditions:

$$\begin{cases} \frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i \\ \frac{\partial L}{\partial w_0} = 0 \Rightarrow 0 = \sum_{i=1}^N \alpha_i y_i \\ \forall i = 1..N, \alpha_i (y_i (w^T x_i + w_0) - 1) = 0 \\ \forall i = 1..N, \alpha_i \geq 0 \end{cases}$$

Let's remark two things:

1. $w = \sum_{i=1}^N \alpha_i y_i x_i$, so w is a linear combination of the x_i
2. $\alpha_i (y_i (w^T x_i + w_0) - 1) = 0$ so at least one of the elements in this product is zero.

So we have two possibilities:

- $\alpha_i > 0$, then $y_i (w^T x_i + w_0) = 1$; x_i is on the margin's boundary
- $\alpha_i = 0$, then x_i is anywhere on the boundary or further... but does not participate in w .

Finding the optimal α_i ? That requires solving the dual problem (obtained by replacing w in the Lagrangian above):

$$\max_{\alpha \in \mathbb{R}^{+N}} L_D(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j$$

such that $\sum_{i=1}^N \alpha_i y_i = 0$

This dual problem is usually solved by a dedicated algorithm called SMO (John Platt (1998), Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines).

Once α is found, $w = \sum_{i=1}^N \alpha_i y_i x_i$ follows, and computing w_0 is almost trivial

We have seen how to compute w but what about w_0 ? Take any support vector x_i , its α_i is non-zero, so $y_i (w^T x_i + w_0) = 1$, so $w_0 = y_i - w^T x_i$. In practice, to avoid numerical errors, one often averages over all support vectors with:

$$w_0 = \frac{\sum_{i \text{ st. } \alpha_i \neq 0} y_i - w^T x_i}{\sum_{i \text{ st. } \alpha_i \neq 0} 1}$$

Overall:

$$w = \sum_{i=1}^N \alpha_i y_i x_i$$

The x_i for which $\alpha_i > 0$ are called **Support Vectors**. They lay on the margin's boundary.

In the end, the optimal separating hyperplane's normal vector is defined as a linear combination of the support vectors' coordinates. This linear combination is **sparse**, it has few non-zero coefficients.

That's it, we've built our first SVM!

✓ 1.4 The non-linearly separable case

Ok, that's all very nice, but what if it is not possible to separate the data with a line, just as in the example below?

```
from sklearn import datasets
from sklearn.utils import shuffle

X1, y1 = datasets.make_gaussian_quantiles(cov=2.0, n_samples=300, n_features=2, n_classes=1)
X1[:,0] = 3. + X1[:,0]
X1[:,1] = 6. + X1[:,1]/2.5
X2, y2 = datasets.make_gaussian_quantiles(cov=1.5, n_samples=300, n_features=2, n_classes=1)
X2[:,0] = 8. + X2[:,0]
X2[:,1] = 4. + X2[:,1]
X3, y3 = datasets.make_gaussian_quantiles(cov=1.5, n_samples=300, n_features=2, n_classes=1)
X3[:,0] = 7. + X3[:,0]
X3[:,1] = 8. + X3[:,1]
X = np.concatenate((X1, X2, X3))
y = np.concatenate((y1, - y2 + 1, y3))
y = 2*y-1
X, y = shuffle(X, y)
Xblue = X[y==-1]
Xred = X[y==1]

# Display
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b',s=20)#, edgecolors='k')
plt.scatter(Xred[:,0],Xred[:,1],c='r',s=20);
```

We still would like to fit a line through the data, since we can assume the overlapping of the red and blue dots is due to noise. In this case, the optimal separating hyperplane can be found by making a **compromise** between the amount of misclassifications and the value of the margin.

For each pair (x_i, y_i) , we introduce the slack variable ξ_i that represents how far x_i is on the wrong side of the margin's boundary, as illustrated by the figure below.



Exercise:

Let's write $y_i(w^T x_i + w_0) \geq M(1 - \xi_i)$ and note that x_i is misclassified if $\xi_i \geq 1$. Reformulate the SVM optimization problem so that we have a maximum "budget" of K misclassifications.

► Ready to see the answer? (click to expand)

This last formulation with K is not satisfying: instead of having to impose a constraint on the number of misclassifications, we would rather make a compromise between the number of misclassifications and the value of the margin. So the optimization problem turns into:

$$\begin{aligned} \min_{w, w_0, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{such that } \forall i = 1..N, \\ & y_i (w^T x_i + w_0) \geq 1 - \xi_i, \\ & \xi_i \geq 0 \end{aligned}$$

Note that C is the parameter that controls the trade-off between having a large margin and having few misclassified points.

This is again a quadratic programming problem. Using the α_i and μ_i Lagrange multipliers, the KKT conditions yield:

$$L_P = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i (y_i (w^T x_i + w_0) - (1 - \xi_i)) - \sum_{i=1}^N \mu_i \xi_i$$

KKT conditions :

$$\frac{\partial L_P}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i$$

$$\begin{aligned}\frac{\partial L_P}{\partial w_0} &= 0 \Rightarrow 0 = \sum_{i=1}^N \alpha_i y_i \\ \frac{\partial L_P}{\partial \xi} &= 0 \Rightarrow \alpha_i = C - \mu_i \\ \forall i = 1..N, \alpha_i (y_i (w^T x_i + w_0) - (1 - \xi_i)) &= 0 \\ \forall i = 1..N, \mu_i \xi_i &= 0 \\ \forall i = 1..N, \alpha_i \geq 0, \mu_i &\geq 0\end{aligned}$$

The dual problem is almost the same as previously (it eliminates all variables except for the α_i) and can be solved using SMO:

$$\begin{aligned}\max_{\alpha \in \mathbb{R}^{+N}} L_D(\alpha) &= \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j \\ \text{such that } \sum_{i=1}^N \alpha_i y_i &= 0 \\ \text{and } 0 \leq \alpha_i &\leq C\end{aligned}$$

Let's follow the same reasoning as previously on the $\alpha_i (y_i (w^T x_i + w_0) - (1 - \xi_i)) = 0$ constraint:

- $\alpha_i > 0$, then $y_i (w^T x_i + w_0) = 1 - \xi_i$: x_i is a **support vector**.
Among the support vectors:
 - $\xi_i = 0$, then $0 < \alpha_i \leq C$
 - $\xi_i > 0$, then $\alpha_i = C$ (because $\mu_i = 0$, because $\mu_i \xi_i = 0$)
- $\alpha_i = 0$, then x_i does not participate in w .

1.5 Making predictions

So, suppose we solve this optimization problem and get (w, w_0) , how do we make a prediction on a new data point x ? Simple! Just compute $w^T x + w_0$: if it's positive we are on one side of the classifier, otherwise we are on the other side.

2. Support Vector Machines in scikit-learn

Fortunately for us, this optimization problem is solved in three lines in scikit-learn.

```
from sklearn import svm
mySVC = svm.SVC(kernel='linear', C=1)
mySVC.fit(X,y)

# Compute margin and find support vectors
w = mySVC.coef_[0]
w0 = mySVC.intercept_
M = 1./np.linalg.norm(w)
print("SV per class:", mySVC.n_support_)
print("w_i:", w)
print("w_0:", w0)
print("Margin:", M)
print("w^T x0 + w_0:", np.dot(w,mySVC.support_vectors_[0,:])+w0)
print("w^T x1 + w_0:", np.dot(w,mySVC.support_vectors_[1,:])+w0)
print("w^T x2 + w_0:", np.dot(w,mySVC.support_vectors_[2,:])+w0)

# Plot the separating plane, the margin and the Support Vectors
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b',s=20)
plt.scatter(Xred[:,0],Xred[:,1],c='r',s=20)
XX = np.arange(-1.,12.,0.1)
YY = -(w[0]*XX+w0)/w[1]
plt.plot(XX,YY,'g')
YY = -(w[0]*XX+w0+1)/w[1]
plt.plot(XX,YY,'g--')
YY = -(w[0]*XX+w0-1)/w[1]
plt.plot(XX,YY,'g--')
plt.scatter(mySVC.support_vectors_[0,0], mySVC.support_vectors_[0,1], s=80, edgecolors='k', facecolors='none');
```

Exercise

Try different values for C in the code above to see how the margin's boundaries evolve.

Remark: since the optimization problem behind SVMs requires the knowledge of the full dataset, SVMs are necessarily an offline method.

Let's plot how the number of support vectors change with C .

```

print("Recall, there are", Xblue.shape[0], "blue points and", Xred.shape[0], "red points.")
logC = np.arange(-4,7,1)
nbSV = np.zeros(len(logC))
for i in range(len(logC)):
    print("Training at C =", 10.**logC[i])
    mySVC = svm.SVC(kernel='linear', C=10.**logC[i])
    mySVC.fit(X,y)
    nbSV[i] = np.sum(mySVC.n_support_)

fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.plot(logC,nbSV)
plt.xlabel("log C")
plt.ylabel("nb support vectors");

```

Let's make a few predictions.

```

mySVC = svm.SVC(kernel='linear', C=1)
mySVC.fit(X,y)

# Compute margin and find support vectors
w = mySVC.coef_[0]
w0 = mySVC.intercept_
M = 1./np.linalg.norm(w)
print("SV per class:", mySVC.n_support_)
print("w_i:", w)
print("w_0:", w0)
print("Margin:", M)
print("w^T x0 + w_0:", np.dot(w,mySVC.support_vectors_[0,:])+w0)
print("w^T x1 + w_0:", np.dot(w,mySVC.support_vectors_[1,:])+w0)
print("w^T x2 + w_0:", np.dot(w,mySVC.support_vectors_[2,:])+w0)

# Plot the separating plane, the margin and the Support Vectors
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b',s=20)
plt.scatter(Xred[:,0],Xred[:,1],c='r',s=20)
XX = np.arange(-1.,12.,0.1)
YY = -(w[0]*XX+w0)/w[1]
plt.plot(XX,YY,'g')
YY = -(w[0]*XX+w0+1)/w[1]
plt.plot(XX,YY,'g--')
YY = -(w[0]*XX+w0-1)/w[1]
plt.plot(XX,YY,'g--')

# Testing data
Xtest = np.array([[2,4],[6,2],[10,2]])
Ypred = mySVC.predict(Xtest)
for i in range(Xtest.shape[0]):
    print("Prediction in", Xtest[i,:], "=", Ypred[i])

plt.scatter(Xtest[:,0],Xtest[:,1], c='m', edgecolors="k", s=80);

```

✓ 3. When using linear separators makes no more sense

Let's take a look at the following data. Would it make sense to try to separate the red points from the blue ones with a straight line?

```

X1, y1 = datasets.make_gaussian_quantiles(cov=2.,
                                          n_samples=300, n_features=2,
                                          n_classes=2, random_state=1)
X2, y2 = datasets.make_gaussian_quantiles(mean=(3, 3), cov=1.5,
                                          n_samples=700, n_features=2,
                                          n_classes=2, random_state=1)

X = np.concatenate((X1, X2))
y = np.concatenate((y1, - y2 + 1))
y = 2*y-1

X, y = shuffle(X, y)

Xtest,X = np.split(X,[400])
ytest,y = np.split(y,[400])

Xblue = X[y==1]
Xred = X[y==0]
fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
plt.scatter(Xblue[:,0],Xblue[:,1],c='b')
plt.scatter(Xred[:,0],Xred[:,1],c='r');

```

It does not look like a linear separation even makes sense. But let's try nonetheless, some optimal separating hyperplane must exist (even if it does not have great generalization properties).

Exercise:

Use scikit-learn to compute a linear Support Vector Classifier for the data above. Compute the classification score and conclude on the limits of this approach.

```
# %load solutions/code3.py
### WRITE YOUR CODE HERE
# If you get stuck, uncomment the line above to load a correction in this cell (then you can execute this code).
```

Quite unconvincing, isn't it?

A short metaphor before we go into more details



Take two blank sheets of paper. On the first one, draw a set of blue points. On the second, draw some red points. Now put one page on top of the other and crumple them together. The red and blue points are separable (whatever number of points there were on each sheet), it's a certainty, since you put them on purpose on two different pieces of paper (two classes). But if you're given the current coordinates of each point, then building a relevant classifier might not be so easy. Some ideas follow from this intuition:

- Good classification might be feasible if we could build an inverse transformation ("uncrumpling") of the data that makes it linearly separable.
- Good classification functions should be able to approximate a large variety of geometric transformations of the data (this inverse transformation should be able to capture a large variety of crumpling patterns).
- A classification method that can only approximate a few geometrical transformations is prone to miss the structure of the data. We call that *underfitting*, which is due to *high bias* in the family of classification functions.
- Having a large set of uncrumpling patterns (being able to approximate many geometrical transformations of the data) comes at the price of taking the risk of picking the wrong uncrumpling pattern given the finite amount of data at our disposal. We call that *overfitting*, which is due to *high variance* in the family of classification functions.

4. A word on the bias-variance tradeoff

The previous example is unconvincing mainly because we stubbornly look for a separator that has the shape of a straight line. Maybe if we allowed more flexibility, we could get better separators?

This compromise between flexibility and prior knowledge is called the **bias-variance** tradeoff in Machine Learning.

- If I search for a function that fits my data within a very restricted set of functions, I introduce a lot of expert knowledge, and thus a lot of **bias**, only allowing for little **variance** in the functions I might find.
- On the other hand, if I use a very rich family of functions, I abandon the introduction of prior information, allowing for high **variance** and low **bias**.

Although appealing, having high variance is not necessarily a good thing: looking for a function in a set that is just too rich leads to the risk of finding several almost equivalent functions that have great statistical properties but no physical meaning.

By maximizing the margin, SVMs actually try to minimize the variance of the family of functions considered: that is a way of trying to find the "simplest" function possible by minimizing $\|w\|$. This process is called **regularization** in Machine Learning. SVMs perform an L_2 regularization, but other forms of regularization exist and are commonly being used (e.g. L_1 regularization).

The bias-variance tradeoff captures the problem of **overfitting** (due to high variance) versus **underfitting** (due to high bias). Let's try to capture this formally. Assume a supervised learning task where one has data $y = f(x) + \epsilon$, where f is a deterministic function and ϵ is a zero-mean noise with variance σ^2 . The mean square generalization error of a prediction function $\hat{f}$ is:

$$err = \mathbb{E}_x \left(\mathbb{E}_{y \sim f(x) + \epsilon} \left(\left(y - \hat{f}(x) \right)^2 \right) \right) = \mathbb{E}_x (err(x))$$

with:

$$err(x) = \mathbb{E}_{y \sim f(x) + \epsilon} \left(\left(y - \hat{f}(x) \right)^2 \right)$$

Let's write $\mathbb{E} = \mathbb{E}_{y \sim f(x) + \epsilon}$, $f = f(x)$ and $\hat{f} = \hat{f}(x)$ for simplicity.

We have then:

$$\begin{aligned} err(x) &= \mathbb{E} \left[y^2 + \hat{f}^2 - 2y\hat{f} \right] \\ &= \mathbb{E} \left[y^2 \right] + \mathbb{E} \left[\hat{f}^2 \right] - 2\mathbb{E} \left[y\hat{f} \right] \end{aligned}$$

But, by definition of the variance:

- $\mathbb{E}[y^2] = \text{Var}(y) + \mathbb{E}[y]^2 = \sigma^2 + f^2$
- $\mathbb{E}[\hat{f}^2] = \text{Var}(\hat{f}) + \mathbb{E}[\hat{f}]^2$
- and f is deterministic so $\mathbb{E}[y\hat{f}] = f\mathbb{E}[\hat{f}]$.

So:

$$\begin{aligned} \text{err}(x) &= \sigma^2 + f^2 + \text{Var}(\hat{f}) + \mathbb{E}[\hat{f}]^2 - 2f\mathbb{E}[\hat{f}] \\ &= \sigma^2 + \text{Var}(\hat{f}) + \left(f^2 - 2f\mathbb{E}[\hat{f}] + \mathbb{E}[\hat{f}]^2\right) \\ &= \sigma^2 + \text{Var}(\hat{f}) + \left(f - \mathbb{E}[\hat{f}]\right)^2 \\ &= \text{Irreducible error} + \text{Variance} + \text{Bias}^2 \end{aligned}$$

Let's explain each of these terms:

- The irreducible error σ^2 is the noise term in the true relationship that cannot fundamentally be reduced by any learning model.
- The bias $f(x) - \mathbb{E}[\hat{f}(x)]$ measures how far from f the best fit of $\hat{f}$ is.
- The variance $\mathbb{E}[\hat{f}^2] - \mathbb{E}[\hat{f}]^2$ of $\hat{f}$ measures how much $\hat{f}$ is able to move around its mean.

So, overall, our goal is to reduce both variance and bias to minimize the generalization error. But in a world with finite data and no information on the true shape of f , there is a compromise to make between minimizing the bias or the variance.

SVMs make this compromise through the C parameter: small C will give more importance to maximizing the margin, thus minimizing the variance (and conversely, large C values will emphasize the data fitting term and thus will minimize the bias).

5. The kernel trick

5.1 The intuition of mapping to higher dimensions

Let's get back to the idea of giving more flexibility to our separator, since hyperplanes did not convince us they could always do the job.

To get around this problem, let's first introduce an illustrative example. The data below come from a voltage test in electronics. They indicate if a component fails or not under a certain voltage (U).

```
U = np.array([[0.3, 0.7, 1.1, 1.8, 2.5, 3.0, 3.3, 3.5, 3.7]]).T
Good = np.array([[ -1,  -1,  -1,  1,  1,  1,  1,  -1,  -1]]).T
plt.figure()
plt.scatter(U, np.zeros((U.shape[0],1)), c=Good, cmap = plt.cm.coolwarm);
```

Question:

Does it look like these data are linearly separable?

Some well-experienced engineer already knows that U is no good criterion to split the data points into two categories with a single threshold (recall: in dimension 1, a hyperplane is a threshold). He also knows that other indicators like $V = (U - 2)^2$ are not any better, but that the pair (U, V) actually allows to build a separating line between points.

```
V = (2-U)**2
plt.figure()
plt.scatter(U, V, c=Good, cmap = plt.cm.coolwarm)
XX = np.arange(np.min(U), np.max(U), 0.05)
YY = -.8+.8*XX
plt.plot(XX,YY, 'g');
```

Our nine points are now linearly separable. But we have not introduced any additional information!

Actually, the only thing we have done, is to *map* our non-separable points x into **another description space**, of higher dimension, where a linear separator exists.

5.2 The kernel trick

Suppose we happen to know such a relevant mapping for our data $\varphi(x) = x'$, where φ maps to a p -dimensional Euclidean space ($p \gg n$, possibly infinite). Then we can compute the optimal linear separator in the corresponding higher dimension description space, find its parameters w' and w'_0 , and whenever we need to make a new prediction in a point x , we first compute its image $x' = \varphi(x)$ and then calculate $w'^T x' + w'_0$ to know on which side of the hyperplane we stand.

But recall that:

$$w' = \sum_{i=1}^N \alpha_i y_i x'_i$$

So :

$$\begin{aligned} w'^T x' + w'_0 &= \left(\sum_{i=1}^N \alpha_i y_i x'_i \right)^T \varphi(x) + w'_0 \\ &= \sum_{i=1}^N \alpha_i y_i \varphi(x_i)^T \varphi(x) + w'_0 \end{aligned}$$

Suppose that, instead of providing us with a mapping $\varphi(x) = x'$, somebody gave us a function $K(x_1, x_2)$ that takes two points x_1 and x_2 , computes their respective images $\varphi(x_1)$ and $\varphi(x_2)$ and returns the dot product:

$$K(x_1, x_2) = \varphi(x_1)^T \varphi(x_2)$$

Then making predictions would boil down to:

$$w'^T x' + w'_0 = \sum_{i=1}^N \alpha_i y_i K(x_i, x) + w'_0$$

The interesting thing is that we don't need to compute φ anymore. The function K is known as a **kernel function** and that's what we call the **kernel trick**.

It is actually possible to compute the α_i and w'_0 just using K and never φ , so, as long as somebody insures that **the kernel K is a dot product in some other descriptor space** we can compute the optimal separating hyperplane of our data in this space, **without ever requiring a knowledge of this descriptor space and the mapping φ** .

5.3 Positive definite kernels

The main question is thus: when can we guarantee that K is an acceptable kernel? It is so if it is an inner product on a (separable) Hilbert space. So in more general words, we are interested in positive, definite kernels on a Hilbert space:

Exercise (off-class):

Check for yourself that the dual optimization problem for the SVM in the kernel's Hilbert space can be written:

$$\begin{aligned} \max_{\alpha \in \mathbb{R}^{+N}} L_D(\alpha) &= \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j) \\ \text{such that } &\sum_{i=1}^N \alpha_i y_i = 0 \\ &\text{and } 0 \leq \alpha_i \leq C \end{aligned}$$

So its resolution only requires the knowledge of K .

$K(\cdot, \cdot)$ is a positive definite kernel on X if

$$\forall n \in \mathbb{N}, x \in X^n \text{ and } c \in \mathbb{R}^n, \sum_{i,j=1}^n c_i c_j K(x_i, x_j) \geq 0$$

We will admit that Mercer's condition below is a sufficient condition for K to be a positive definite kernel:

Given $K(x, y)$, if:

$$\forall g(x) / \int g(x)^2 dx < \infty, \iint K(x, y) g(x) g(y) dx dy \geq 0$$

Then, there exists a mapping $h(\cdot)$ such that:

$$K(x, y) = \langle h(x), h(y) \rangle$$

There are many kernels that have been developped in the litterature. Combined with what we have seen before, it allows to build non-linear SVMs. The nice thing is that some kernels actually map our data to a descriptor space of infinite dimension, where it is presumably a lot easier to find a separating hyperplane.

One such kernel is the so-called "radial basis kernel" which is very popular and can be written:

$$K(x_1, x_2) = e^{-\gamma \|x_1 - x_2\|^2}$$

Other common kernels:

- polynomial $K(x, y) = (1 + \langle x, y \rangle)^d$
- sigmoid $K(x, y) = \tanh(\kappa_1 \langle x, y \rangle + \kappa_2)$

6. SVMs and kernels

Let's practice on the last dataset with a radial basis kernel.

```
# Plot separator, margin and support vectors
def plot_SVC(mySVC):
    fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
    plt.scatter(Xblue[:,0],Xblue[:,1],c='b')
    plt.scatter(Xred[:,0],Xred[:,1],c='r')
    XX, YY = np.meshgrid(np.arange(np.min(X[:,0]),np.max(X[:,0]),0.1), np.arange(np.min(X[:,1]),np.max(X[:,1]),0
    ZZ = mySVC.decision_function(np.c_[XX.ravel(), YY.ravel()])
    ZZ = ZZ.reshape(XX.shape)
    plt.contour(XX, YY, ZZ, levels=[0],alpha=0.75)
    fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
    cont = plt.contour(XX, YY, ZZ, levels=[-1., 0., 1.], alpha=0.75)
    plt.xlabel(cont, cont.levels, inline=True, fontsize=10)
    fig=plt.figure(figsize=fig_size, dpi= 80, facecolor='w', edgecolor='k')
    cont = plt.contourf(XX, YY, ZZ, alpha=0.75, cmap = plt.cm.coolwarm)

# We can play with C
mySVC = svm.SVC(kernel='rbf', C=1.)
mySVC.fit(X,y)

print("SV per class:", mySVC.n_support_)
plot_SVC(mySVC)
```

A lot better isn't it? Let's play a little with the value of C .

Exercise:

Let's see how that SVM with the rbf kernel would do on the data from the beginning of this notebook.

```
# %load solutions/code4.py
### WRITE YOUR CODE HERE
# If you get stuck, uncomment the line above to load a correction in this cell (then you can execute this code).
```

Exercise:

And on the "linearly separable + noise" case?

```
X1, y1 = datasets.make_gaussian_quantiles(cov=2.0, n_samples=300, n_features=2, n_classes=1)
X1[:,0] = 3. + X1[:,0]
X1[:,1] = 6. + X1[:,1]/2.5
X2, y2 = datasets.make_gaussian_quantiles(cov=1.5, n_samples=300, n_features=2, n_classes=1)
X2[:,0] = 8. + X2[:,0]
X2[:,1] = 4. + X2[:,1]
X3, y3 = datasets.make_gaussian_quantiles(cov=1.5, n_samples=300, n_features=2, n_classes=1)
X3[:,0] = 7. + X3[:,0]
X3[:,1] = 8. + X3[:,1]
X = np.concatenate((X1, X2, X3))
y = np.concatenate((y1, - y2 + 1, y3))
y = 2*y-1
X, y = shuffle(X, y)
Xblue = X[y== -1]
Xred = X[y==1]

# Display
mySVC = svm.SVC(kernel='rbf')
mySVC.fit(X,y)
print("SV per class:", mySVC.n_support_)
plot_SVC(mySVC);
```

These two last experiments illustrate that introducing complex kernels such as the rbf one is sometimes detrimental.

✓ 7. What about other uses?

SVMs can be adapted to deal with **multi-class** classification by training "one-versus-one" or "one-versus-rest" binary SVMs. In the first case, for C classes, $C(C-1)/2$ classifiers need to be built and a majority vote is implemented. In the second case, only C classifiers are built, at the risk of class imbalance.

SVMs can also be used to solve **regression** problems, although they are not often used for that in practice. The optimization problem differs a little but the main ideas remain. In scikit-learn, they can be accessed by using the `svm.SVR` class.

In the regression case, we do not want to separate anymore but to *fit* the hyperplane to the data. For this purpose, we define a *loss function* $\sum_i V(y_i - f(x_i))$ and the objective function becomes:

$$\min_{w, w_0} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N V(y_i - w^T x_i + w_0)$$

Usual loss functions:

- ϵ -insensitive, $V(z) = \begin{cases} 0 & \text{if } |z| \leq \epsilon \\ |z| - \epsilon & \text{otherwise} \end{cases}$
- Laplacian, $V(z) = |z|$
- Gaussian, $V(z) = \frac{1}{2} z^2$
- Huber's robust loss, $V(z) = \begin{cases} \frac{1}{2\sigma} z^2 & \text{if } |z| \leq \sigma \\ |z| - \frac{\sigma}{2} & \text{otherwise} \end{cases}$

The ϵ -insensitive loss:



The optimization problem and its resolution are quite similar to the classification case (but a little heavier, notation-wise) and left to your curiosity, just as the extension to the automation of the search for the trade-off parameter C , through ν -SVR.

Similarly, one can derive a formulation of SVMs for **density estimation** by using *One-Class SVMs*. If you are curious to know more, please refer to the references mentioned a few lines below.

SVMs easily support class weights and sample weights and can deal with unbalanced problems.

The general description of [SVMs in scikit-learn](#) is a good reminder on the versatility and the limitations of SVMs.

In a nutshell:

- Support Vector Machines for classification try to separate data by maximizing a geometrical margin
- They are computed offline
- They offer a sparse, robust to class imbalance, and easy to evaluate predictor
- Kernels are a way of enriching (lifting) the data representation so that it becomes linearly separable
- SVMs + kernels offer a versatile method for classification, regression and density estimation
- [Documentation in scikit-learn](#)

References

On the general theory of SVMs for classification:

A tutorial on Support Vector Machines for Pattern Recognition.

C. J. C. Burges, *Data Mining and Knowledge Discovery*, **2**, 131-167, (1998).

On support vector regression (and its extension to ν -SVR):

A tutorial on Support Vector Regression.

A. J. Smola and B. Schölkopf, *Journal of Statistics and Computing*, **14**(3), 199-222, (2004).

New support vector algorithms. B. Schölkopf, A. J. Smola, R. C. Williamson, and P. L. Bartlett. *Neural computation*, **12**(5), 1207-1245, (2000).

On One-Class SVMs:

Support vector method for novelty detection.

B. Schölkopf, R. C. Williamson, A. J. Smola, J. Shawe-Taylor, and John C. Platt. *Neural Information Processing Systems*, **12**, 582-588, (1999).

On multi-class SVMs:

On the algorithmic implementation of multiclass kernel-based vector machines.

K. Crammer and Y. Singer. *Journal of machine learning research*, **2**, 265-292, (2001).

8. Examples

8.1 Spam or ham?

Exercise:

Your turn to play: let's build a spam classifier using the ling-spam dataset and a linear SVM. Compute it's score on the validation dataset below.

```
from sys import path
path.append('../2 - Text data preprocessing')
import load_spam
spam_data = load_spam.spam_data_loader()
spam_data.load_data()
```

```
Xtrain, ytrain, Xtest, ytest = spam_data.split(2000)
```

```
print(Xtrain.shape)
print(Xtest.shape)
```

```
# %load solutions/code5.py
### WRITE YOUR CODE HERE
# If you get stuck, uncomment the line above to load a correction in this cell (then you can execute this code).
```

Exercise:

We've trained our model in the Tf-Idf data. Let's see how the model behaves on raw word counts.

```
Xtrain, ytrain, Xtest, ytest = spam_data.split(2000, feat='wordcount')
```

```
# %load solutions/code6.py
### WRITE YOUR CODE HERE
# If you get stuck, uncomment the line above to load a correction in this cell (then you can execute this code).
```

Exercise: Want to try another kernel than linear?

Commencez à coder ou à [générer](#) avec l'IA.

Let's keep the tf-idf with linear SVM classifier (which seems to work better) and use it to identify which are the misclassified emails (and find the confusion matrix by the way).

```
# Retrain
Xtrain, ytrain, Xtest, ytest = spam_data.split(2000)
spam_svc = svm.SVC(kernel='linear', C=1.)
spam_svc.fit(Xtrain,ytrain);
```

```
# Find misclassified examples
ypredict = spam_svc.predict(Xtest)
misclass = np.not_equal(ypredict, ytest)
Xmisclass = Xtest[misclass,:]
ymisclass = ytest[misclass]
misclass_indices = [i for i, j in enumerate(misclass) if j == True]
print("Misclassified messages indices:", misclass_indices)
```

```
from sklearn.metrics import confusion_matrix
print("Confusion matrix:")
print(confusion_matrix(ytest, ypredict))
```

```
# Check some misclassified mails
index = misclass_indices[0]+2000
print("Prediction:", spam_svc.predict(spam_data.tfidf[index,:]))
spam_data.print_email(index)
```

8.2. NIST

Let's evaluate SVMs on the optical character recognition task of the NIST data.

```
from sklearn import datasets
%matplotlib inline
import matplotlib.pyplot as plt

digits = datasets.load_digits()
print(digits.data.shape)
print(digits.images.shape)
print(digits.target.shape)
print(digits.target_names)

X = digits.data
y = digits.target
Xtrain,Xtest = np.split(X,[1000])
ytrain,ytest = np.split(y,[1000])
#Xtrain = X[:1000,:]
#ytrain = y[:1000]
```

```
#Xtest = X[1000:,:]
#ytest = y[1000:]

print(digits.DESCR)

plt.gray();
plt.matshow(digits.images[0]);
plt.show();
plt.matshow(digits.images[15]);
plt.show();
plt.matshow(digits.images[42]);
plt.show();
```

```
from sklearn.utils import shuffle

def shuffle_and_split(X,y,n):
    X0,y0 = shuffle(X,y)
    Xtrain,Xtest = np.split(X0,[n])
    ytrain,ytest = np.split(y0,[n])
    return Xtrain, ytrain, Xtest, ytest
```

```
print(Xtrain.shape)
print(ytrain.shape)
digits_svc = svm.SVC(kernel='rbf', gamma=1e-3)
digits_svc.fit(Xtrain,ytrain)
prediction = digits_svc.predict(Xtest)
#print("Training error:", np.sum(np.not_equal(prediction,ytest))/len(ytest))
print("Generalization error:", np.sum(np.not_equal(prediction,ytest))/len(ytest) )
print("Generalization score:", digits_svc.score(Xtest,ytest))
```

```
# Compute cross-validation score
nb_trials = 20
score = []
for i in range(nb_trials):
    Xtrain, ytrain, Xtest, ytest = shuffle_and_split(X,y,1000)
    digits_svc = svm.SVC(kernel='rbf', gamma=1e-3)
    digits_svc.fit(Xtrain,ytrain)
    score += [digits_svc.score(Xtest,ytest)]
    print('*',end='')
print(" done!")
```

```
print("Average generalization score:", np.mean(score))
print("Standard deviation:", np.std(score))
```

Let's identify which are the misclassified images (and find the confusion matrix by the way).

```
# Retrain
Xtrain = X[:1000,:]
ytrain = y[:1000]
Xtest = X[1000:,:]
ytest = y[1000:]
digits_svc = svm.SVC(kernel='rbf', gamma=1e-3)
digits_svc.fit(Xtrain,ytrain);
```

```
# Examples
N = 1053
plt.matshow(digits.images[N])
plt.show()
x = digits.data[N,:]
print("prediction on image number", N, ":", digits_svc.predict([digits.data[N,:]]))
print("correct label", digits.target[N])
```

```
# Find misclassified examples
ypredict = digits_svc.predict(Xtest)
misclass = np.not_equal(ypredict, ytest)
Itest = digits.images[1000:,:]
Xmisclass = Xtest[misclass,:]
ymisclass = ytest[misclass]
Imisclass = Itest[misclass,:]
```

```
print("Confusion matrix:")
print(confusion_matrix(ytest, ypredict))
```

```
# Display misclassified examples
N = 1
```

```
plt.matshow(Xmisclass[N])
plt.show()
print("prediction on image number", N, ":", digits_svc.predict([Xmisclass[N,:]]))
print("correct label", N, ":", ymisclass[N])
```

8.3 : ionosphere dataset

This dataset contains data collected by a radar system in Goose Bay, Labrador. This system consists of a phased array of 16 high-frequency antennas designed to detect free electrons in the ionosphere. In general, there are two types of structures in the ionosphere: "Good" and "Bad". These radars detected these structures and passed the signal.

[Download from UCI](#)

Questions :

- How many variable are there ? Which are independent and which are dependent ?
- Is it a good dataset to try classification on ?
- Using SVM, try to obtain at least 64% of classification performance (top is 94%). Note : justify your result with k-fold cross